

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms

- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code

The complete implementation of the Single Layer Perceptron experiment, including the source code, dataset details, execution instructions, and dependencies, is available in the GitHub repository.

GitHub Repository:

<https://github.com/VIKASNI-S/Deep-Learning-Lab/blob/main/Lab-1/Lab-1%20Sorce%20Code.ipynb>

1 Mandatory Plots

Plot	Purpose	Expected Inference
Feature Histograms	Distribution Analysis	Identify skewness and outliers.
Correlation Heatmap	Feature Relationship	Observe highly correlated features.
Scatter Plot	Class Distribution	Determine whether classes are approximately linearly separable.
Training Error vs Epoch	Learning Behaviour	Verify convergence of the perceptron.
Weight Evolution	Parameter Analysis	Observe stabilization of learned weights.
Bias Evolution	Parameter Analysis	Understand the role of the bias term.
Confusion Matrix	Performance Evaluation	Interpret TP, FP, TN and FN.
Learning Rate Comparison	Hyperparameter Study	Compare convergence behaviour for different learning rates.
Decision Boundary (Optional)	Visualization	Illustrate the separating hyperplane using two selected features.

Histograms of Numerical Features

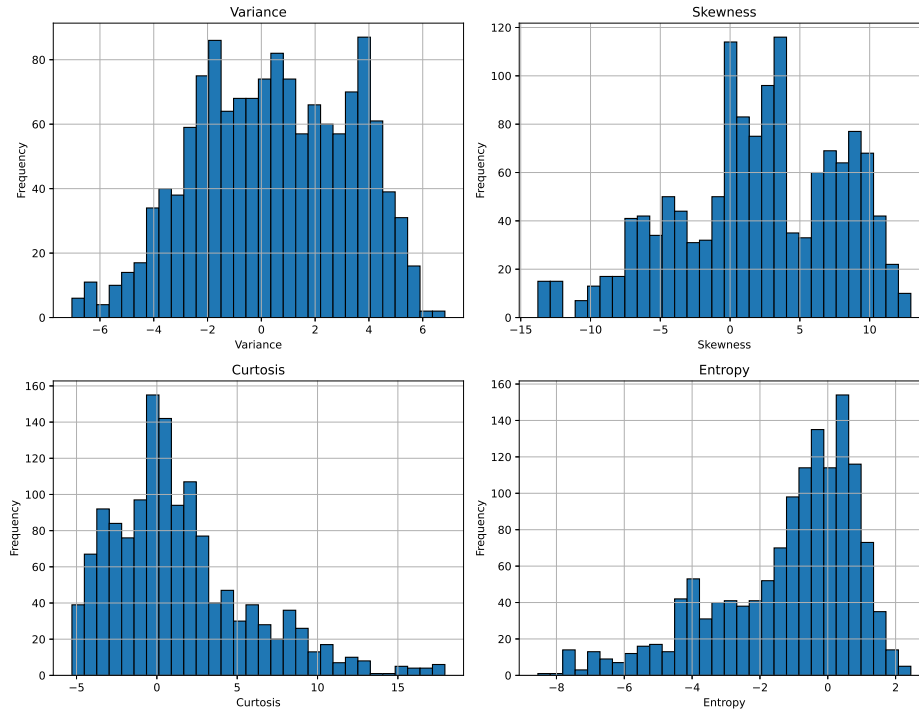


Figure 1: Feature Histograms

The histograms show the distribution of the four banknote features: Variance, Skewness, Kurtosis, and Entropy. The features exhibit different ranges and distributions, with some showing skewness and a wider spread of values. This indicates that normalization is necessary before training the perceptron.

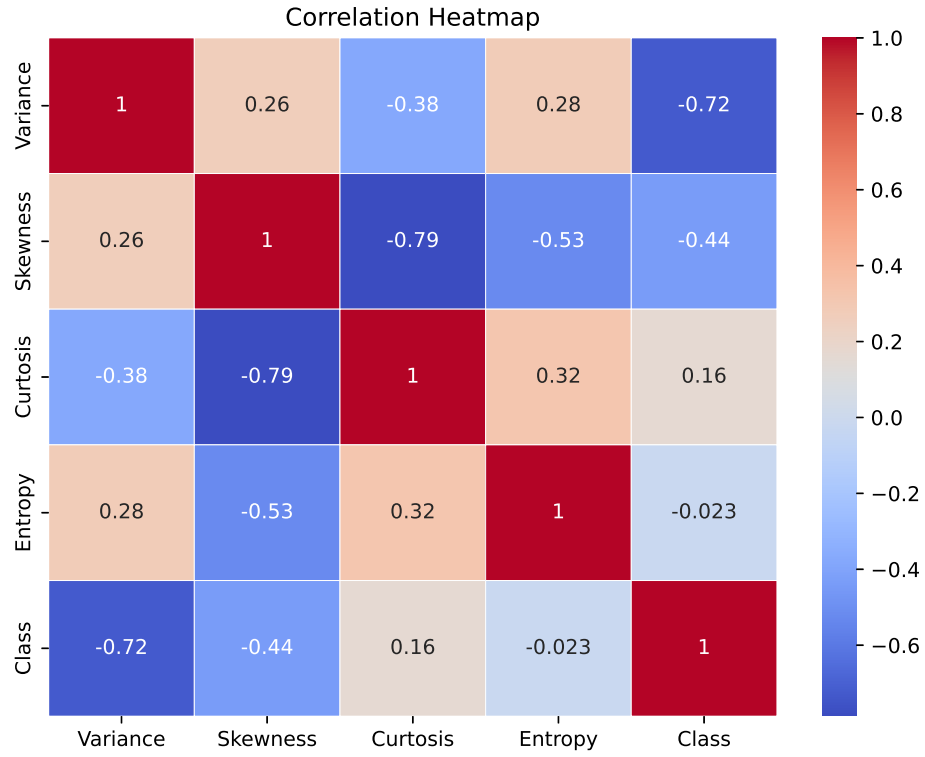


Figure 2: Correlation Heatmap

The correlation heatmap reveals the relationships among the four extracted image features of the banknotes. Some features show strong positive or negative correlations, indicating that they contain related information useful for distinguishing genuine and forged banknotes. The target class also shows noticeable correlation with several features, suggesting their importance in classification.

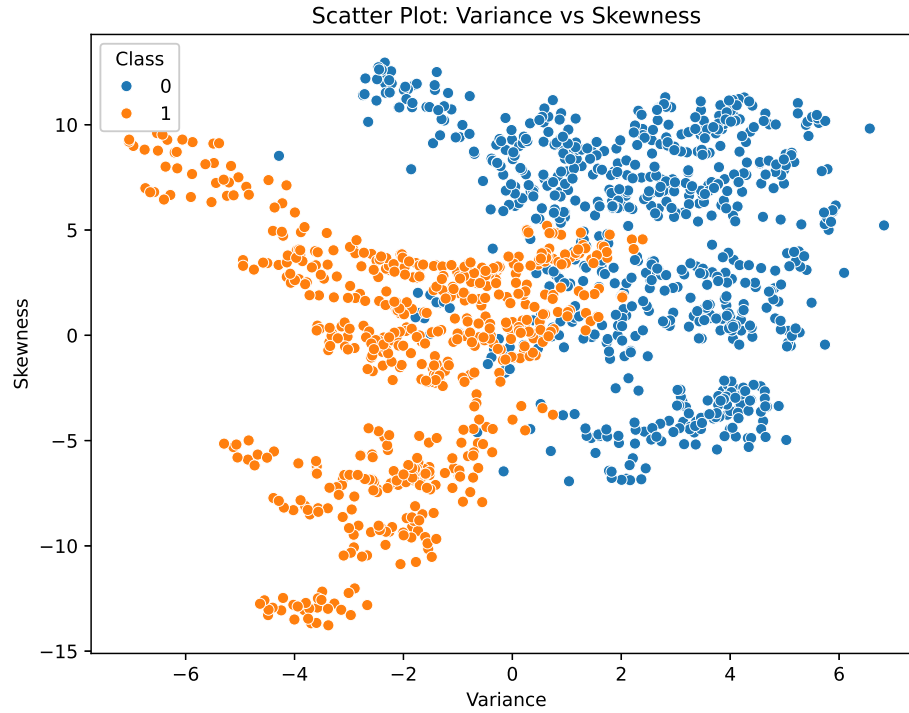


Figure 3: Scatter Plot

The scatter plot of Variance and Skewness shows that genuine and forged banknotes form distinct clusters with only slight overlap. This indicates that these features provide good discriminatory information and that the dataset is approximately linearly separable, making it suitable for a single-layer perceptron.

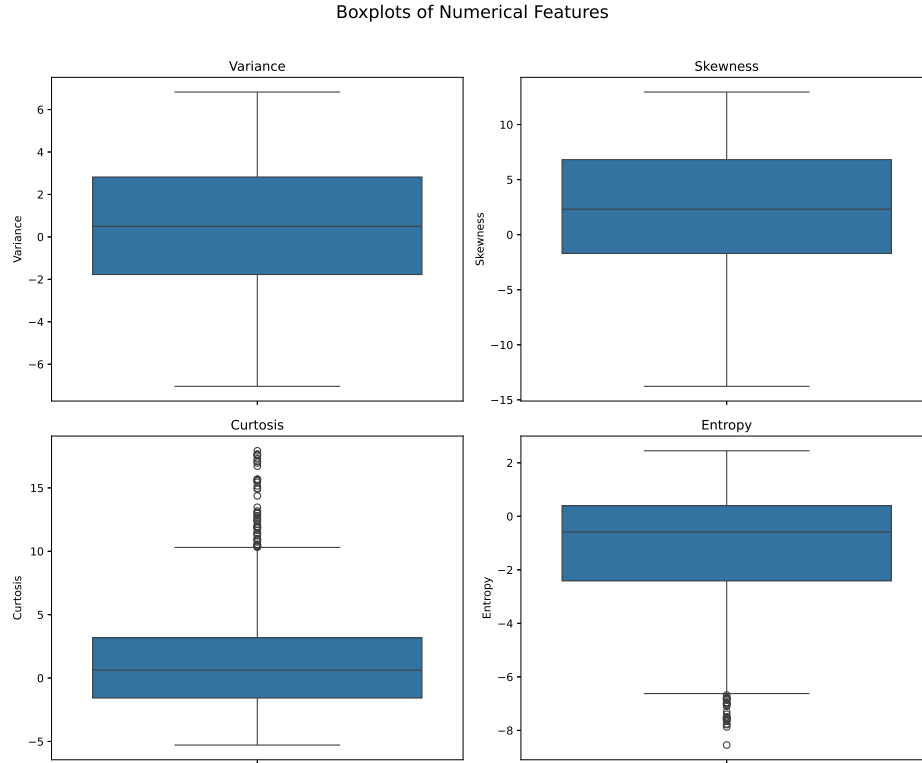


Figure 4: Box Plot

The box plots illustrate the distribution, spread, and presence of outliers for each feature in the Banknote Authentication dataset. Variance, Skewness, Kurtosis, and Entropy exhibit several outliers, indicating the presence of extreme observations. Despite these outliers, the overall feature distributions are well-defined, suggesting that the dataset is suitable for training the Perceptron model after appropriate preprocessing such as normalization.

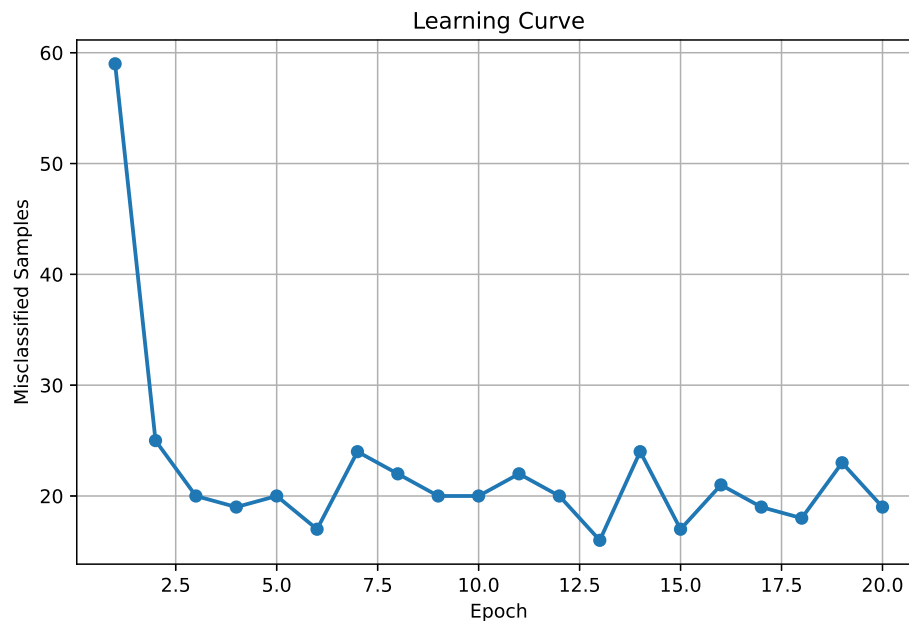


Figure 5: Training Error vs Epoch

The number of misclassified banknote samples generally decreased as the epochs increased, showing that the perceptron successfully learned the classification pattern. A small fluctuation in misclassified samples was observed during training due to continuous weight updates, which is a normal characteristic of the perceptron learning algorithm. Overall, the decreasing trend indicates successful convergence.

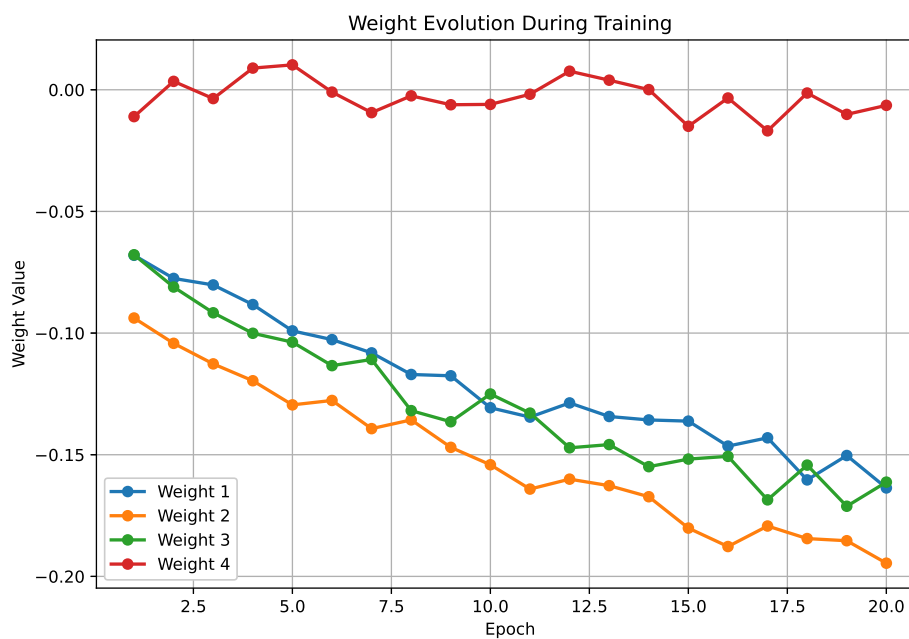


Figure 6: Weight Evolution

The weights corresponding to the banknote features changed significantly during the initial epochs as the perceptron learned the importance of each feature. As training progressed, the weight values gradually stabilized, indicating convergence toward an optimal linear decision boundary for classifying genuine and forged banknotes.

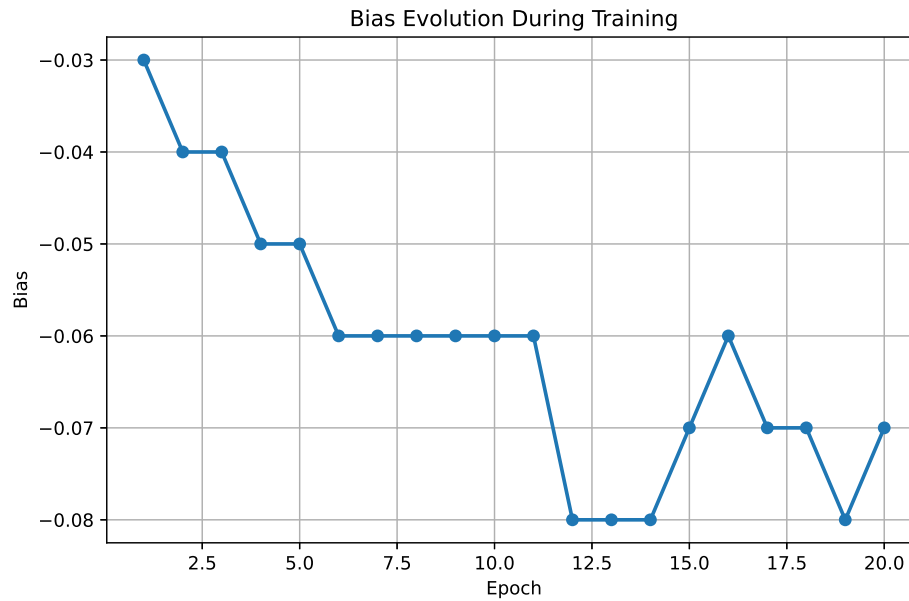


Figure 7: Bias Evolution

The bias value was updated throughout training to improve the position of the decision boundary. After several epochs, the bias gradually stabilized, indicating that the perceptron had learned an appropriate threshold for distinguishing between genuine and forged banknotes.

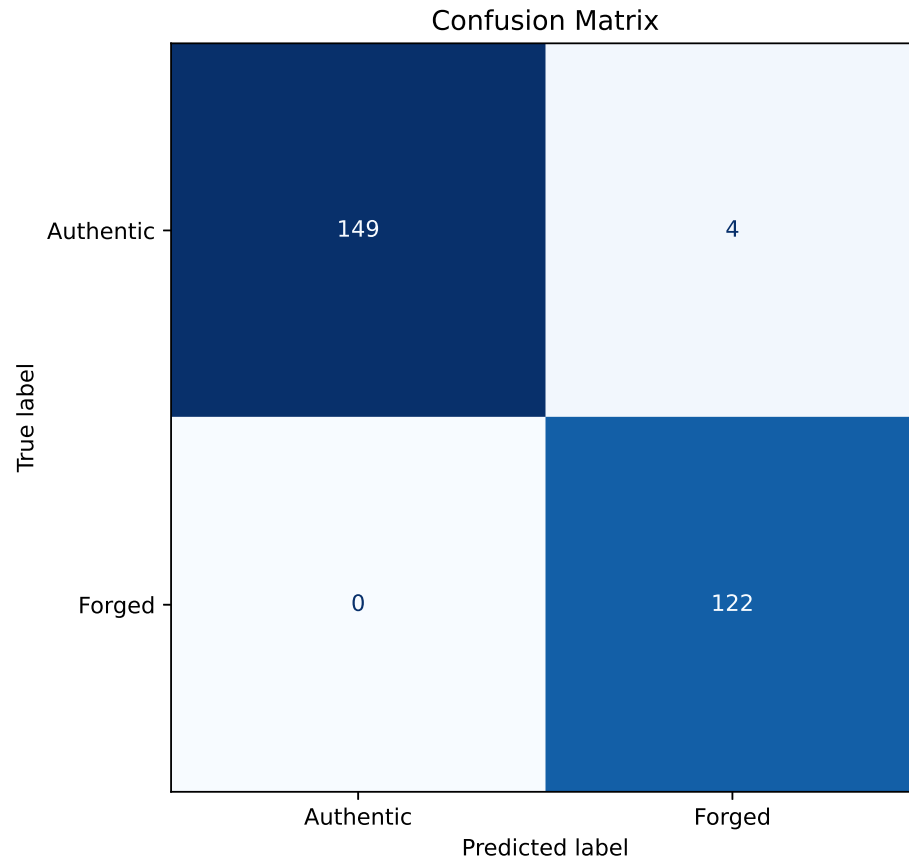


Figure 8: Confusion Matrix

The confusion matrix shows that the majority of genuine and forged banknotes were classified correctly, with only a few misclassifications. This demonstrates that the perceptron effectively learned the characteristics of the Banknote Authentication dataset and achieved good classification performance on unseen test data.

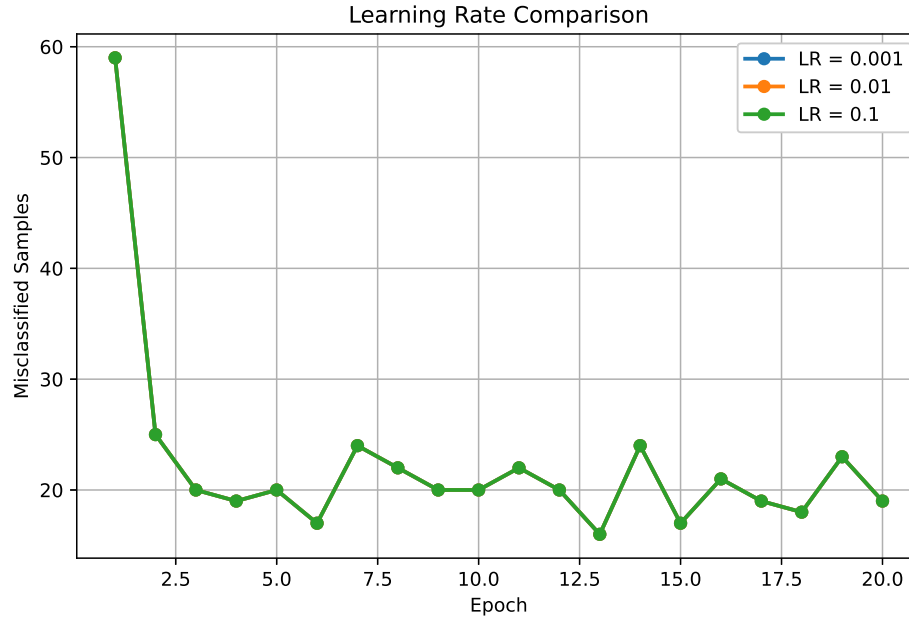


Figure 9: Learning Rate Comparison

The learning rate comparison demonstrates how different learning rates influence the convergence of the perceptron on the Banknote Authentication dataset. Smaller learning rates resulted in smoother but slower convergence, whereas larger learning rates reached convergence faster with slight fluctuations. An appropriate learning rate provided efficient and stable learning for this dataset.

8. Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	80:20
Learning Rate	0.01
Epochs	20
Final Weights	[-0.16364416, -0.19457442, -0.16126111, -0.00640573]
Final Bias	-0.0700
Accuracy	0.9855
Precision	0.9683
Recall	1.0
F1-score	0.9839

Epoch	Errors	W1	W2	W3	W4	Bias
1	59	-0.068025	-0.093809	-0.067819	-0.011036	-0.030
2	25	-0.077536	-0.104214	-0.081067	0.003453	-0.040
3	20	-0.080213	-0.112661	-0.091664	-0.003621	-0.040
4	19	-0.088281	-0.119595	-0.100081	0.008909	-0.050
5	20	-0.099090	-0.129506	-0.103721	0.010233	-0.050
6	17	-0.102683	-0.127692	-0.113383	-0.000959	-0.060
7	24	-0.108134	-0.139275	-0.110844	-0.009393	-0.060
8	22	-0.117019	-0.135681	-0.131877	-0.002491	-0.060
9	20	-0.117554	-0.146960	-0.136437	-0.006154	-0.060
10	20	-0.130691	-0.154124	-0.125042	-0.006007	-0.060
11	22	-0.134531	-0.164116	-0.132884	-0.001838	-0.060
12	20	-0.128706	-0.160035	-0.147151	0.007633	-0.080
13	16	-0.134286	-0.162712	-0.145837	0.003939	-0.080
14	24	-0.135704	-0.167261	-0.154905	0.000066	-0.080
15	17	-0.136189	-0.180152	-0.151786	-0.015007	-0.070
16	21	-0.146447	-0.187732	-0.150660	-0.003383	-0.060
17	19	-0.143050	-0.179299	-0.168548	-0.016878	-0.070
18	18	-0.160345	-0.184462	-0.154263	-0.001339	-0.070
19	23	-0.150317	-0.185344	-0.171170	-0.010111	-0.080
20	19	-0.163644	-0.194574	-0.161261	-0.006406	-0.070

Table 2: Epoch-wise Learning Progress

9. Additional Tasks

1. Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.
2. Compare the implementation with Scikit-learn's Perceptron.

3. Repeat the experiment using learning rates 0.001, 0.01 and 0.1.
4. Explain why a Single Layer Perceptron cannot solve the XOR problem.
5. Study the effect of feature normalization on model convergence.

9.1 Comparison of Step and Sigmoid Activation Functions

Table 3: Comparison of Step and Sigmoid Activation Functions

Feature	Step Activation	Sigmoid Activation
Output Range	0 or 1	0 to 1
Nature	Binary	Continuous and Smooth
Differentiable	No	Yes
Gradient	Undefined at threshold	Well-defined
Suitable for Backpropagation	No	Yes
Typical Usage	Classical Perceptron	Neural Networks

The Step Activation Function produces binary outputs (0 or 1), making it suitable for simple binary classification tasks. However, it is not differentiable, preventing the computation of gradients required during training. Since modern deep learning relies on gradient-based optimization methods such as Gradient Descent and Adam, the Step Activation Function cannot be effectively used.

The Sigmoid Activation Function is smooth and differentiable, allowing gradients to be computed for backpropagation. Although it suffers from the vanishing gradient problem in deep networks, it is far more suitable for neural network training than the Step Activation Function.

9.2 Comparison with Scikit-learn's Perceptron

Table 4: Comparison of Custom Perceptron and Scikit-learn Perceptron

Aspect	Custom Implementation	Scikit-learn Perceptron
Weight Update	Manual implementation	Automatic implementation
Learning Algorithm	User-defined	Optimized implementation
Ease of Use	Moderate	Easy
Performance	Educational	Production-ready
Additional Features	Limited	Regularization, Shuffling, Early Stopping

The custom implementation helps understand the mathematical concepts behind the Perceptron Learning Algorithm. In contrast, Scikit-learn provides an optimized implementation with additional features such as automatic convergence checks, data shuffling, regularization, and efficient computation. Both implementations achieve similar classification performance when trained using the same dataset and hyperparameters.

9.3 Effect of Different Learning Rates

The experiment was repeated using learning rates of 0.001, 0.01, and 0.1. The observations are summarized in Table 5.

Table 5: Effect of Learning Rate on Training

Learning Rate	Observation
0.001	Slow learning and requires more epochs for convergence.
0.01	Stable convergence with balanced weight updates.
0.1	Faster convergence but may cause oscillations before reaching the optimum solution.

A small learning rate (0.001) updates the weights slowly, increasing the training time. A moderate learning rate (0.01) provides stable convergence and consistent learning. A larger learning rate (0.1) accelerates convergence but may produce fluctuations due to larger weight updates. For the Banknote Authentication dataset, a learning rate of 0.01 provided the best balance between convergence speed and stability.

9.4 Why a Single Layer Perceptron Cannot Solve the XOR Problem

A Single Layer Perceptron can classify only linearly separable datasets because it learns a single linear decision boundary. The XOR dataset is not linearly separable, meaning no single straight line can correctly separate all the data points into their respective classes.

Therefore, regardless of the number of training epochs, a Single Layer Perceptron cannot correctly classify the XOR dataset. Solving the XOR problem requires a Multi-Layer Perceptron (MLP) with one or more hidden layers and nonlinear activation functions such as Sigmoid or ReLU. This limitation of the Single Layer Perceptron motivated the development of modern deep neural networks.

9.5 Effect of Feature Normalization on Model Convergence

Feature normalization scales all input features to a similar numerical range before training.

Table 6: Effect of Feature Normalization

Without Normalization	With Normalization
Slower convergence	Faster convergence
Uneven weight updates	Balanced weight updates
Requires more epochs	Requires fewer epochs
Less stable learning	More stable learning

Without normalization, features with larger numerical values dominate the learning process, leading to slower and less stable convergence. After normalization, all features contribute equally to weight updates, resulting in faster convergence and smoother optimization. Although the final classification accuracy remains nearly unchanged for the Banknote Authentication dataset, normalization significantly improves training efficiency and convergence stability.

10. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.